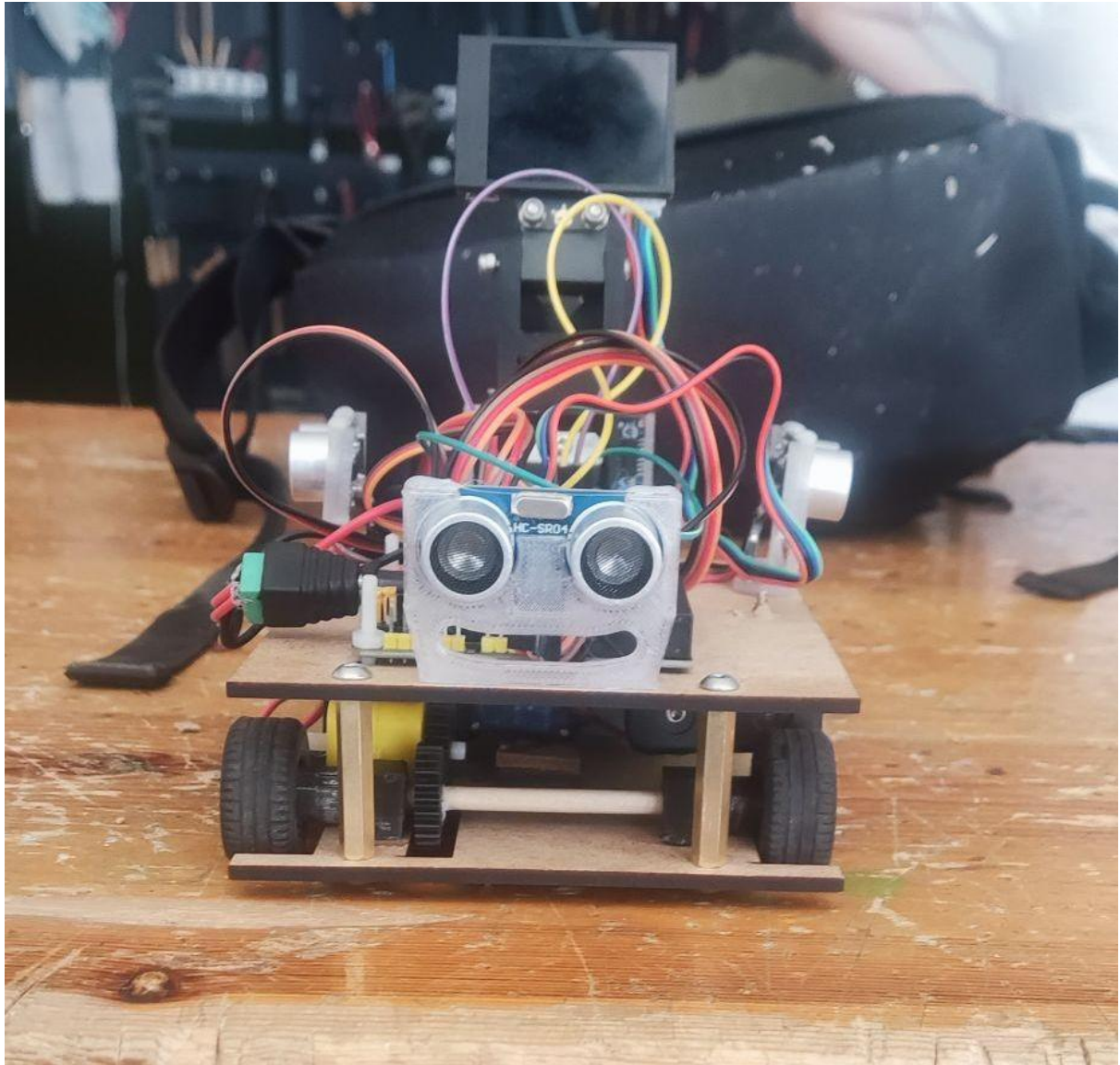


MEMORIA TÉCNICA: WRO FUTURE ENGINEERS



Contenido

1. NUESTRO PROYECTO.....	3
2. INTRODUCCIÓN Y ESTRATEGIA	3
2.1 Resumen de componentes y programa	3
2.2 Objetivos	3
3. DISEÑO MECÁNICO (HARDWARE)	4
3.1 Estructura del chasis	4
3.2 Sistema de dirección	4
3.3 Sistema de tracción	4
3.4 Distribución de componentes.....	4
4. ELECTRÓNICA Y SENSORES.....	4
4.1 Controlador principal	4
4.2 Sensores de visión	4
4.3 Sensores de distancia y orientación	5
4.4 Baterías	5
5. SOFTWARE (PROGRAMACIÓN).....	5
5.2 Uso de la Cámara IA Huskylens	5
5.3 Diagrama de flujo	5
5.4 Código	6
5.5 Explicación	12
1. Atras	13
2. Adelante	13
3. color_verde	13
4. sinobjetos	13
5. color_rojo	13
6. parar	13
6. PRUEBAS, ITERACIONES Y RESULTADOS.....	13
6.1 Proceso de pruebas	14
6.2 Problemas encontrados y soluciones aplicadas.....	14
6.3 Tiempos y rendimiento	14
7. CONCLUSIONES Y TRABAJO FUTURO	14
7.1 Lecciones aprendidas.....	14
7.2 Mejoras futuras.....	14
8. Componentes y piezas.....	14
8.1 Lista de materiales y justificación.....	14

8.2 Justificación de la elección de los componentes:.....	16
8.3 Planos y diseños CAD y referencias:.....	18
9. ANEXOS, REFERENCIAS Y FOTOS	18

1. NUESTRO PROYECTO

Título del proyecto: THE BEST FAILURES Categoría: WRO Future Engineers Nombre del equipo: Colegio Nuestra Señora de las Mercedes Integrantes: Sergio, Isaac y Martín Centro: Colegio Nuestra Señora de las Mercedes-Herencia- Ciudad Real Año: 2026

2. INTRODUCCIÓN Y ESTRATEGIA

Hemos construido nuestro robot utilizando los componentes que le dieron a nuestro colegio por el programa ESCUELA 4.0. Utilizando este material junto con la impresora 3D y la cortadora láser hemos construido todas las piezas.

2.1 Resumen de componentes y programa

Hemos utilizado una microbit con una expansión (Shield) de KeyStudio, a la que hemos conectado una cámara con IA Huskylens, un servo SG90 de 180° para controlar la dirección de cremallera hecha con Lego, un motor DC con reductora puesto para mover el eje trasero, 3 sensores de ultrasonido Hc-sr04, un controlador de motor L298N, un portabaterías para dos baterías 18650 y un interruptor. El robot es capaz de desplazarse de forma que recorra un circuito entre paredes y también evitar obstáculos y pasar por la derecha si ve prismas rojo y por la izquierda si los ve verde.

2.2 Objetivos

- Diseñar un robot ligero utilizando los componentes que tenemos.
- Empezar por que el robot se mueva.
- Intentar dar vueltas al circuito.
- Si es posible esquivar las piezas de colores por su lado.

3. DISEÑO MECÁNICO (HARDWARE)

3.1 Estructura del chasis

El chasis es DM de 3mm, y lo hemos diseñado con inkscape y luego cortado en la cortadora láser. Tiene una parte inferior donde va el motor, la dirección, las baterías, el servo, la dirección y una parte superior en la que están el resto de componentes. También lleva piezas diseñadas en 3D como los engranajes, soporte para la cámara, ejes y sensores de ultrasonido, también lleva tornillos y piezas de la caja de expansión de la microbit.

Peso total: 1,2kg

Dimensiones: [120 x 180 x 150 mm]

3.2 Sistema de dirección

La dirección es de tipo Ackerman, aunque realmente es cremallera de Lego, lleva un servomotor SG-90 9G de 180°.

3.3 Sistema de tracción

Motor: Motor DC con reductora incorporada 1:48

Relación de marchas: 1:1 (los dos engranajes son iguales)

Tipo de tracción: Trasera

3.4 Distribución de componentes

Las baterías y el motor en la parte inferior pretenden mantener equilibrado el chasis, hemos colocado los componentes para que no tenga más peso en ningún lado y no influya en su movimiento.

4. ELECTRÓNICA Y SENSORES

4.1 Controlador principal

Microbit V.2. Es fácil de programar por bloques y teníamos en nuestro colegio. Se pueden conectar a través de la tarjeta de expansión la cámara y varios sensores.

4.2 Sensores de visión

Cámara utilizada: Huskylens V.1. Compatible con microbit y dispone de una extensión que facilita su uso. La utilizamos para detectar colores, el color rojo y el verde.

4.3 Sensores de distancia y orientación

Sensores de distancia: el robot lleva 3 sensores de ultrasonidos HC-SR04, para detectar las paredes y que no se choque. No hemos podido poner uno en la parte trasera porque no teníamos más pines para conectar.

Giroscopio: La microbit lleva giroscopio y brújula incorporada, pero no hemos conseguido programarlas bien.

4.4 Baterías

Baterías usadas: 2xNCR18650B Li-ion MH12210 Panasonic 3400 mAh 3.7v y los reguladores de voltaje los lleva la Sensor Shield Keystudio v2.0 for Microbit

5. SOFTWARE (PROGRAMACIÓN)

5.1 Programación en la nube en makecode, utilizado en vez de librerías extensiones, las extensiones utilizadas han sido las siguientes: Huskylens V.1, Servo y Sónar. Programación por bloques tipo Scratch.

5.2 Uso de la Cámara IA Huskylens

Al usar la extensión de Huskylens, lo primero que debemos hacer es iniciar la cámara por conectada por I2C y seleccionar el modo de trabajo a reconocimiento de color. Detectamos 3 colores Id1 color rojo, Id2 color verde e Id3 color de aparcamiento. No detectamos líneas en el suelo.

5.3 Diagrama de flujo


```

    pins.analogWritePin(AnalogPin.P1, 600)
    basic.pause(tiempo)
    pins.digitalWritePin(DigitalPin.P1, 0)
}
// inicio al boton A, repite la funcion de las rondas 1
// y 2 o la 3 y 4(con objetos)
input.onButtonPressed(Button.A, function () {
    c = huskylens.readBox_s(Content3.ID)
    if (c <= 0) {
        while (true) {
            sinobjetos()
        }
    } else {
        while (true) {
            objetos()
        }
    }
})
// programa para las rondas 3 y 4
function objetos () {
    // definimos variables
    c = huskylens.readBox_s(Content3.ID)
    posX = huskylens.readBox_index(1, 1,
Content1.xCenter)
    huskylens.writeName(1, "Rojo")
    df = sonar.ping(
DigitalPin.P16,
DigitalPin.P15,
PingUnit.Centimeters
)
}

```

```
dd = sonar.ping(  
    DigitalPin.P13,  
    DigitalPin.P14,  
    PingUnit.Centimeters  
)  
di = sonar.ping(  
    DigitalPin.P12,  
    DigitalPin.P11,  
    PingUnit.Centimeters  
)  
if (c <= 0) {  
    // cuando se acerque a la pared  
    if (df < 26) {  
        // comprueba la dirección donde deba girar  
        if (dd > di) {  
            basic.showIcon(IconNames.House)  
            parar(100)  
            servos.P0.setAngle(55)  
            Atras(600)  
            parar(100)  
            servos.P0.setAngle(125)  
            Adelante(300)  
        } else {  
            basic.showIcon(IconNames.Cow)  
            parar(100)  
            servos.P0.setAngle(125)  
            Atras(600)  
            parar(100)  
            servos.P0.setAngle(55)  
            Adelante(300)  
        }  
    }  
}
```



```

    }
  } else {
    // se mantiene en un punto intermedio de las
paredes

    if (dd > di || 10 > di) {
      parar(50)
      servos.P0.setAngle(125)
      Adelante(200)
    } else if (dd < di || 10 > dd) {
      parar(50)
      servos.P0.setAngle(55)
      Adelante(200)
    }
  }
} else if (c == 1) {
  color_rojo()
} else if (c == 2) {
  color_verde()
}
}

function color_verde () {
  if (posx >= 160) {
    servos.P0.setAngle(110)
    Atras(500)
    servos.P0.setAngle(55)
    Adelante(400)
  } else {
    servos.P0.setAngle(110)
    Atras(500)
    servos.P0.setAngle(55)
  }
}

```

```

        Adelante(400)
    }
}
function sinobjetos () {
    df = sonar.ping(
        DigitalPin.P16,
        DigitalPin.P15,
        PingUnit.Centimeters
    )
    dd = sonar.ping(
        DigitalPin.P13,
        DigitalPin.P14,
        PingUnit.Centimeters
    )
    di = sonar.ping(
        DigitalPin.P12,
        DigitalPin.P11,
        PingUnit.Centimeters
    )
    // quitar la diferencia e igualación a 0 cuando haya
un centro en el tablero
    if (df < 25) {
        if (dd > di) {
            basic.showIcon(IconNames.House)
            parar(100)
            servos.P0.setAngle(55)
            Atras(600)
            parar(100)
            servos.P0.setAngle(110)
            Adelante(300)
        }
    }
}

```

```

    } else {
        basic.showIcon(IconNames.Cow)
        parar(100)
        servos.P0.setAngle(110)
        Atras(600)
        parar(100)
        servos.P0.setAngle(55)
        Adelante(300)
    }
} else {
    if (dd > di || 10 > di) {
        servos.P0.setAngle(110)
    } else if (dd < di || 10 > dd) {
        servos.P0.setAngle(55)
    }
    Adelante(100)
}
}

// cuando vea el rojo
function color_rojo () {
    if (posx <= 160) {
        servos.P0.setAngle(55)
        Atras(500)
        servos.P0.setAngle(110)
        Adelante(400)
    } else {
        servos.P0.setAngle(55)
        Atras(500)
        servos.P0.setAngle(110)
        Adelante(400)
    }
}

```

```

    }
}
function parar (t: number) {
    pins.analogWritePin(AnalogPin.P1, 0)
    basic.pause(t)
}
let di = 0
let dd = 0
let df = 0
let posX = 0
let c = 0
servos.P0.setAngle(90)
c = 0
huskylens.initI2c()
huskylens.initMode(protocolAlgorithm.ALGORITHM_COLOR_RECOGNITION)
pins.digitalWritePin(DigitalPin.P2, 0)
pins.digitalWritePin(DigitalPin.P8, 1)
loops.everyInterval(500, function () {
    huskylens.request()
})

```

5.5 Explicación

La forma en la que el robot toma decisiones es bastante simple: primero mira qué está detectando con sus sensores y con la cámara HuskyLens, y según eso decide qué hacer. Es como si tuviera “modos”. Si no ve colores especiales, entra en el modo normal y usa los sensores ultrasónicos para saber si tiene un obstáculo delante o si está demasiado cerca de una pared. Cuando detecta algo muy cerca, retrocede y gira hacia el lado donde haya más espacio. Si no hay obstáculos, intenta mantenerse centrado entre las paredes moviendo el servo un poco hacia la izquierda o la derecha.

Además, el robot también reacciona a los colores que detecta la cámara. Si ve rojo o verde, cambia a otro modo y hace una maniobra especial: gira el servo, retrocede y luego avanza

hacia el lado que corresponde. Es como si el color le diera una orden rápida para cambiar de dirección.

Tiene 6 funciones:

1. Atras

Esta función hace que el robot vaya hacia atrás durante el tiempo que tú le digas. Básicamente enciende los pines del motor en la dirección contraria y le mete potencia. Luego espera el tiempo indicado y para el motor. Es como decirle: “retrocede un poco y luego quieto”.

2. Adelante

Esta es lo mismo, pero al revés: el robot avanza. Activa los pines para que el motor gire hacia adelante y le da fuerza. Después de avanzar el tiempo que le pongas, hace unas maniobras extra (girar servo, retroceder, etc.) según lo que esté pasando. Es como su movimiento principal.

3. color_verde

Cuando la cámara detecta el color verde, el robot hace una maniobra especial: gira el servo, retrocede y luego avanza hacia un lado. Da igual si el verde está a la izquierda o derecha, porque en tu código hace lo mismo siempre. Es como si dijera: “¡He visto verde, toca esquivar!”.

4. sinobjetos

Esta función es la que usa cuando no hay colores detectados. Aquí el robot mira las distancias con los sensores ultra sonidos. Si tiene algo delante, decide hacia qué lado girar según dónde haya más espacio. Si no hay nada, solo corrige su dirección para mantenerse centrado. Es como su modo “normal”.

5. color_rojo

Funciona igual que la del color verde, pero para el rojo. Cuando lo ve, retrocede, gira el servo y avanza hacia el lado contrario. Es como una reacción rápida para evitar algo marcado en rojo.

6. parar

Esta es la función más simple: corta la potencia del motor y espera un momento. Sirve para que el robot haga pausas entre movimientos y no se vuelva loco.

6. PRUEBAS, ITERACIONES Y RESULTADOS

6.1 Proceso de pruebas

No disponemos de pista de pruebas y hemos simulado en la casa de Isaac, en la cocina, solo disponíamos de un cuadrado de 1x1 m y las paredes de la cocina. Parece que puede dar algunas vueltas y evitar algún obstáculo.

6.2 Problemas encontrados y soluciones aplicadas

Problema 1: Pequeño incendio en portabaterías al ponerlas mal.

Solución: Reemplazo de portabaterías

Problema 2: No funcionaban los sensores de ultrasonidos.

Solución: Problema de pines, algunos pines están reservados y los usábamos para los sensores, tuvimos que mirar los pines que podíamos utilizar.

Problema 3: Se quemó el servo.

Solución: Cambio de servo.

Problema 4: El robot gira mal hacia la izquierda.

Solución: Seguimos pensando en ello.

6.3 Tiempos y rendimiento

En la competición veremos lo que hace nuestro robot.

7. CONCLUSIONES Y TRABAJO FUTURO

7.1 Lecciones aprendidas

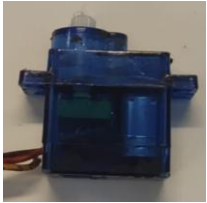
Hemos aprendido a ser constantes y trabajar, aunque las cosas muchas veces no salen bien. Si pensamos los 3 las cosas salen mejor que si las piensa 1. También hemos aprendido que hay que pensar y contar hasta 10 antes de encender el robot.

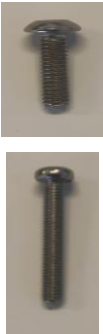
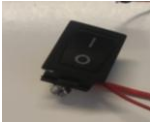
7.2 Mejoras futuras

- Cambiar la dirección, porque no va recto cuando queremos.
- Motor y eje trasero integrados.
- Chasis más resistente.
- Si queremos usar 4 sensores de ultrasonidos utilizar otra placa.

8. Componentes y piezas.

8.1 Lista de materiales y justificación.

Componentes	Cantidad	Precio	Imagen
Tabla 40x40cm dm 3mm	1	2 €	
Motor DC con reductora	1	Disponible en el cole.	
Servo SG90 9G 180°	1	Disponible en el cole.	
L298N	1	Disponible en el cole.	
Piezas Lego dirección	-	Disponible en el cole.	
Portabaterías 18650	2	3,09 €	
Baterías Li-ion Panasonic MH12210	2	Disponible en el cole.	
Shield Keystudio V.2	1	9,91 €	
Hc-sr04	3	Disponible en el cole.	
Cámara Huskylens V.1	1	Disponible en el cole.	

Ruedas Lego 30.4x14	4	Disponible en el centro.	
Placa Micro:bit V2	1	Disponible en el centro	
Tornillería varios			
Conector DC	1	0,70 €	
Interrupto	1	Disponible en el centro	

8.2 Justificación de la elección de los componentes:

Componentes:

- Motor de corriente continua con reductora.

Hemos utilizado un motor con reductora ya que es un motor versátil ya que puede ir hacia adelante y hacia atrás modificando su velocidad además de hemos podido enganchar los engranajes para que pueda avanzar y retroceder

- L298N, para poder utilizar el motor.

Hemos utilizado el L298N ya que el consumo del motor es tan grande que la placa micro bit no lo soporta por lo que lo necesita para controlar cuál será la velocidad y sentido del movimiento

- Placa microbit + Shield Keyestudio

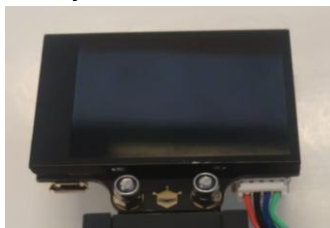


Hemos elegido la placa micro bit por su gran compatibilidad con la huskylens y porque es fácil de programar

- Portabaterías 18650 LIPO 2 en serie.

Hemos utilizado un portapilas que proporciona la energía suficiente para que todo funcione bien y para que el motor de corriente continua obtenga la suficiente energía para que pueda funcionar

- Husky Lens V1, cámara IA



Utilizamos la husky lens para poder reconocer los colores y así poder realizar la segunda prueba con mayor facilidad

- HC-SR04

Lo hemos utilizado para poder medir distancias y poder hacer que se mueva con mayor precisión a la hora de la primera prueba y poder mantener distancias con los bordes negros además de que mejoran la percepción de los obstáculos del robot

- Servo 180°.

Hemos utilizado el servo 180 para poder tener mayor manejo a la hora de poder girar con la cremallera con piezas lego y así permitir que vaya a izquierda y derecha

- Ruedas Lego

Hemos utilizado ruedas de lego ya que podíamos imprimir en 3D piezas que encajen con el sistema de movimiento y porque al no tener un gran tamaño podemos estructurar el espacio de forma más eficiente sin olvidar que al estar hechas de goma su fricción es mayor lo que hace que los movimientos del robot sean más precisos.

8.3 Planos y diseños CAD y referencias:

Chasis 1: chasis robot1.svg

Chasis 2: chasis robot2.svg

Soporte Sensores de ultrasonido: diseño 3D "Escornabot Brivoi Compactus", creado por el usuario escornabot en 2016, está disponible en Thingiverse bajo licencia CC BY-SA 4.0. escornabot, "Escornabot Brivoi Compactus," Thingiverse, 2016. Disponible en: [thingiverse.com](https://www.thingiverse.com/thing/1111111).

Soporte Huskylens: El diseño 3D "huskylens Ai camera mount for FTC", creado por el usuario e_2249455 (Team Fusion 18142) en 2024, está disponible en Printables.com. Creative Commons Atribución-NoComercial 4.0 Internacional (CC BY-NC 4.0). e_2249455. (2024). Huskylens Ai camera mount for FTC [Modelo 3D]. Printables. <https://www.printables.com/model/942759-huskylens-ai-camera-mount-for-ftc>
Archivos: "husky lens mount part 1.stl", "husky lens mount part 2.stl"

Acople servo eje lego: creado por nosotros en Tinkercad.
Archivo: "acople lego servo.stl"

Acople eje 5mm a rueda lego: creado por nosotros en Tinkercad.
Archivo: "Acople lego 5mm.stl"

Engranajes: Modificación del diseño 3D "Engranajes para usar con Servo Sg90", creado por el usuario jjnm5 en 2015, está disponible en Thingiverse bajo licencia CC BY 4.0. jjnm5. (2015). Engranajes para usar con Servo Sg90 [Modelo 3D]. Thingiverse. <https://www.thingiverse.com/thing/797477>

9. ANEXOS, REFERENCIAS Y FOTOS

- Repositorio público en GitHub: [enlace](#)
- **Fotografías del robot (desde todos los ángulos) y una fotografía del equipo.**
- Vídeo demostrativo mostrando el robot funcionando de manera autónoma en las diferentes regiones

[Misión 1](#)

[Misión 3](#)

